LAB REPORT

Lab Title	Arrays	Lab No.	06
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/4/7		

Lab description/objectives:

1. Demonstrate understanding of two-dimensional arrays and array manipulation in C programming.
2. Implement the functionality of finding the largest element in each row, calculating the transpose of a square matrix and performing symmetry Judgment as defined in the lab tasks/requirements.
3. Analyze critical aspects.

Source code:

Task I:

```
#include <stdio.h>
int M;
void maxRow(int a[M][M],int b[M]){
    for (int i = 0; i < M; i++){
        b[i] = a[i][0];
        for (int j = 1; j < M; j++){
            if (a[i][j] > b[i]){
                b[i] = a[i][j];
            }
        }
    }
}
int main(){
    printf("Please enter the size of the square matrix.\n");
    scanf("%d", &M);
    int a[M][M], b[M];
    printf("Please enter the element of the matrix.\n");
    for (int i = 0; i < M; i++){
        for (int j = 0; j < M; j++){
```

```
            scanf("%d", &a[i][j]);
        }
    }
    maxRow(a, b);
    for (int i = 0; i < M; i++){
        printf("The largest element in the row %d is %d\n", i, b[i]);
    }
    return 0;
}
```

Task II:

```
#include <stdio.h>
int N;
void transpose(int a[N][N], int b[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            b[j][i] = a[i][j];
        }
    }
}
int isSymmetric(int a[N][N]){
    int ret = 1;
    for (int i = 0; i < N; i++){
        for (int j = i + 1; j < N; j++){
            if (a[i][j] != a[j][i]){
                ret = 0;
                return ret;
            }
        }
    }
    return ret;
}
```

```
int main(){
    printf("Please enter the size of the square matrix.\n");
    scanf("%d", &N);
    int a[N][N], b[N][N];
    printf("Please enter the elements in the matrix.\n");
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            scanf("%d", &a[i][j]);
        }
    }
    transpose(a, b);
    printf("The transpose of the matrix is:\n");
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            printf("%d ", b[i][j]);
        }
        printf("\n");
    }
    if (isSymmetric(a)){
        printf("The matrix is symmetric.\n");
    }else{
        printf("The matrix is not symmetric.\n");
    }
    return 0;
}
```

Program output:

Task I:

```
Please enter the size of the square matrix.
3
Please enter the element of the matrix.
1 3 4
2 5 5
8 7 1
The largest element in the row 0 is 4
The largest element in the row 1 is 5
The largest element in the row 2 is 8
```

Task II:

```
Please enter the size of the square matrix.
3
Please enter the elements in the matrix.
1 2 4
3 4 2
1 5 6
The transpose of the matrix is:
1 3 1
2 4 5
4 2 6
The matrix is not symmetric.
```

```
Please enter the size of the square matrix.
4
Please enter the elements in the matrix.
1 2 4 5
2 3 2 1
4 2 5 6
5 1 6 3
The transpose of the matrix is:
1 2 4 5
2 3 2 1
4 2 5 6
5 1 6 3
The matrix is symmetric.
```

Analysis:

1. Finding the Largest Element in Each Row

- Clarify the column size of the two-dimensional array when passing the reference to the function so that the compiler can ensure the right address.

- Ensure the iterator will not exceed the limit of the size. That is to control the boundary as less than N and begin at 0.

2. **Transpose of a Matrix and Symmetry Judge**

- Clear the logical sequence when store the transpose of matrix A to B.

- When judging the symmetry, choose the appropriate method to reduce the times of loop. That is just need to check the half of matrix to another half. So we can let the inner loop begin at i+1.

Conclusion：

When using the array, we need to ensure the size of it in advance and notice the boundary when iterating the array. When using the high-dimensional array as the parameter of functions, we need to clarify the necessary size to ensure the compiler can find the right continuous address.